

Relatório de Arquitetura

Implementação de um Data Lakehouse de E-commerce com Databricks e Delta Lake

Aluno: Luis Maio

Disciplina: Implementação de Data Lakehouse com Databricks e Delta Lake
[26E3_3]

Instituto Infnet — 2026

Repositório do código-fonte:
github.com/LuisFelipeMTCS/ecommerce_lakehouse

1. Introdução e Contexto

Este relatório documenta a arquitetura e a implementação de um Data Lakehouse completo para um cenário de e-commerce, construído do zero como um **Databricks Asset Bundle** e implantado em um workspace real (Databricks Free Edition, computação exclusivamente serverless). O objetivo do projeto foi demonstrar, de ponta a ponta e com execução verificável, as competências de arquitetura Lakehouse no padrão Medallion (Bronze/Silver/Gold), usando o framework **Lakeflow Declarative Pipelines** (anteriormente conhecido como Delta Live Tables — DLT), governança centralizada via **Unity Catalog**, e as garantias transacionais do protocolo **Delta Lake**.

Diferente de um protótipo teórico, todo o pipeline descrito aqui foi de fato implantado (*deploy*) e executado no workspace até a conclusão bem-sucedida de todas as suas tarefas. As seções seguintes descrevem a arquitetura, o esquema de transação do Delta Lake, as estratégias de qualidade e governança, as técnicas de otimização aplicadas, e encerram com uma análise interpretativa das métricas de qualidade e desempenho coletadas diretamente do event log do pipeline e do histórico de transações das tabelas Delta.

2. Arquitetura da Solução: Padrão Medallion com Lakeflow Declarative Pipelines

A solução segue o fluxo evolutivo clássico do Lakehouse — ingestão bruta, refino transacional e agregação de negócio — implementado como um único pipeline declarativo (`ecommerce_medallion_dlt`), escrito inteiramente em SQL e composto por três arquivos de biblioteca correspondentes às três camadas:

Camada	Arquivo	Papel
Bronze	<code>src/pipeline/bronze.sql</code>	Ingestão bruta via Auto Loader, schema fixo, sem transformação de negócio
Silver	<code>src/pipeline/silver.sql</code>	Validação (Expectations), quarentena e Change Data Capture (CDC)
Gold	<code>src/pipeline/gold.sql</code>	Agregações de negócio, mascaramento de PII e streaming com watermark

A ingestão é feita via **Auto Loader** (função de tabela `STREAM read_files(...)`), que monitora incrementalmente um volume do Unity Catalog (`/Volumes/ecommerce_lakehouse/bronze/landing`) e processa automaticamente novos arquivos JSON (clientes, pedidos) e CSV (produtos) assim que chegam, sem necessidade de reprocessar o histórico completo a cada execução. Cada streaming table de bronze usa `schemaHints` para fixar o tipo das colunas e captura a coluna de *rescued data* (dados que não se encaixam no schema esperado), monitorada por uma expectation de aviso (WARN).

Item de rubrica atendido: O aluno configurou a captura de dados de forma escalável para tratar novos arquivos automaticamente.

2.1 Orquestração e Automação com Delta Live Tables

Todo o grafo de dependências entre as tabelas (bronze → silver → gold) é resolvido automaticamente pelo framework declarativo: o desenvolvedor descreve o *que* cada tabela deve conter (via `CREATE OR REFRESH STREAMING TABLE` e `CREATE MATERIALIZED VIEW`), e o motor do Lakeflow infere a ordem de execução, paraleliza o que é possível e reexecuta somente o necessário a cada atualização. O pipeline foi configurado como **serverless** (exigência da Free Edition, que não disponibiliza clusters clássicos), com Photon habilitado e modo de execução `continuous: false` (execução sob demanda, disparada por um job).

Item de rubrica atendido: O aluno utilizou o framework Delta Live Tables para automatizar os fluxos e dependências.

2.2 Estrutura do Projeto como Databricks Asset Bundle

Todo o projeto — código do pipeline, jobs, notebooks operacionais e parâmetros de implantação — é versionado como um **Databricks Asset Bundle** (infraestrutura como código), permitindo deploy e execução reproduzíveis via CLI:

```
databricks.yml          # bundle, variables, targets (inclui resources/*.yaml)
resources/
  pipeline_ecommerce.yml # definicao do pipeline Lakeflow
  job_setup_e_carga.yml  # job principal: setup -> dados -> pipeline -> operacao
  job_ingestao.yml       # job por chegada de arquivo no volume (PAUSED)
  job_manutencao.yml     # job agendado (cron) de manutencao noturna (PAUSED)
src/
  setup/                 # catalogo, governanca (UC), gerador de dados
  pipeline/              # bronze.sql, silver.sql, gold.sql
  operacao/              # manutencao, metricas, checkpoint, validacao
docs/workflow/           # config. de referencia (cluster classico c/ autoscale)
```

3. Esquema de Transação do Delta Lake (Protocolo ACID)

Toda tabela do projeto é uma tabela Delta, cujo estado é definido por um *transaction log* (diretório `_delta_log`) — uma sequência ordenada de arquivos JSON, cada um descrevendo as ações atômicas (adição/remoção de arquivos de dados, alteração de metadados, mudança de protocolo) que constituem uma versão da tabela. É esse log, e não o conteúdo dos arquivos Parquet em si, que garante as propriedades **ACID** (Atomicidade, Consistência, Isolamento e Durabilidade): uma transação só é considerada efetivada quando seu arquivo de commit é gravado com sucesso, de forma atômica, no log.

Essa anatomia foi demonstrada de forma prática no notebook de manutenção (`src/operacao/manutencao_delta.sql`), que cria uma tabela de auditoria (`auditoria.pedidos_snapshot`) e, em sequência, aplica exatamente as operações que evidenciam o protocolo de transação:

Versão	Operação	O que demonstra
0	CREATE TABLE AS SELECT	Snapshot inicial materializado a partir de <code>silver.pedidos</code>
1	UPDATE	Reescrita seletiva de arquivos (copy-on-write) para <code>status='ENVIADO'</code> antigos
2	DELETE	Remoção transacional de pedidos cancelados antigos
3	MERGE	Upsert (176 atualizações + 22 inserções) reconciliando com <code>silver.pedidos</code>
4	DELETE	“Acidente” proposital: exclusão de toda a tabela
5	RESTORE TO VERSION	Recuperação do estado anterior ao acidente via <code>time travel</code>
6	OPTIMIZE ZORDER BY	Compactação de arquivos + ordenação física (ver seção 6)
7/8	REORG ... APPLY (PURGE) / VACUUM	Atualização de deletion vectors e arquivos obsoletos

Cada uma dessas operações gera uma nova versão imutável da tabela, consultável via `DESCRIBE HISTORY` — o que também habilita **time travel** (consulta/restauração de qualquer versão anterior). A tabela de evidências (seção 9) mostra o histórico real coletado após a execução, incluindo as métricas de linhas afetadas por cada operação e a contagem de arquivos adicionados/removidos, extraídas diretamente do log de transação via `operationMetrics`.

Item de rubrica atendido: O aluno demonstrou domínio sobre a anatomia da tabela Delta e seu protocolo de transação.

Item de rubrica atendido: O aluno utilizou o versionamento do Delta Lake no projeto para auditoria ou recuperação de estados anteriores.

4. Qualidade de Dados: Expectations e Isolamento de Registros Inválidos

A camada Silver aplica **Expectations** — restrições declarativas de qualidade — em streaming tables intermediárias de "eventos validados", antes de qualquer CDC ser aplicado. Três níveis de severidade foram utilizados, cada um mapeado para uma necessidade de negócio diferente:

Severidade	Cláusula	Uso no projeto
Aviso	EXPECT (...)	Sinaliza sem descartar — ex.: UF ausente em clientes, rescued_data em bronze
Descarte	... ON VIOLATION DROP ROW	Remove silenciosamente do fluxo principal — ex.: CPF/e-mail inválidos
Falha controlada	... ON VIOLATION FAIL UPDATE	Interrompe a atualização — ex.: produto_id nulo, receita negativa

Para pedidos, em vez de simplesmente descartar registros ruins, o pipeline implementa uma **tabela de quarentena** (`silver.pedidos_quarentena`), que replica o mesmo predicado de validação de forma invertida e anexa uma coluna `regras_violadas` (array) indicando exatamente qual(is) regra(s) foi(ram) violada(s) — `pedido_id` nulo, `cliente_id` nulo, `quantidade` ≤ 0 , `preço` ≤ 0 , status inválido ou data de pedido posterior à data do evento. Isso preserva o dado bruto para auditoria futura, em vez de simplesmente descartá-lo.

Item de rubrica atendido: O aluno implementou regras de validação para tratar e isolar dados com falhas de integridade.

5. Captura de Mudanças (Change Data Capture)

Cada entidade de negócio usa `CREATE FLOW ... AS AUTO CDC INTO` para aplicar os eventos validados sobre a tabela final, com uma estratégia de SCD (Slowly Changing Dimension) escolhida de acordo com a necessidade analítica:

Entidade	Estratégia	Chave / Sequência	Motivo
Clientes	SCD Type 2 (histórico completo)	KEYS cliente_id, SEQUENCE BY evento_seq	Preservar histórico de mudanças de endereço/UF ao longo do tempo
Pedidos	SCD Type 1 (estado atual)	KEYS pedido_id, SEQUENCE BY evento_seq	Só o status mais recente do pedido interessa ao negócio
Produtos	SCD Type 1 com DELETE físico	KEYS produto_id, SEQUENCE BY produto_id_desc	Produtos descontinuados são removidos, não versionados

O CDC também resolve corretamente eventos fora de ordem: um evento de atualização de cliente com `evento_seq` anterior ao já aplicado é automaticamente ignorado pelo `SEQUENCE BY`, sem necessidade de lógica adicional — uma garantia importante em cenários reais onde mensagens podem chegar fora de ordem (reprocessamento, múltiplas origens, etc.).

Item de rubrica atendido: O aluno configurou o pipeline para refletir corretamente as mudanças de dados utilizando captura de mudanças (CDC).

6. Governança Centralizada e Isolamento de Dados Sensíveis (Unity Catalog)

6.1 Federação de Identidade e Catalogação Centralizada

Todos os dados do projeto residem em um único catálogo do Unity Catalog (ecommerce_lakehouse), organizado em sete schemas com responsabilidades claras: bronze, silver, gold (as três camadas medallion), referencia (dependência externa), governanca (funções de máscara e row filter), auditoria (snapshots e checkpoints) e monitoramento (métricas do pipeline). O acesso é controlado por três **grupos de identidade federada ao nível de conta** (não apenas do workspace), criados e geridos centralmente:

Grupo	Acesso concedido
ecom_engenharia	Leitura/escrita em bronze, silver e gold + volume de landing (dono do dado)
ecom_pii_leitores	Leitura apenas da camada gold, com PII real (sem máscara)
ecom_analistas	Leitura apenas da camada gold, com PII mascarada e restrita às UFs SP/RJ/MG/GO/DF

O isolamento entre camadas é reforçado por GRANTS explícitos: nem ecom_analistas nem ecom_pii_leitores recebem qualquer privilégio sobre os schemas bronze ou silver — o acesso a dado granular/bruto com PII completa fica restrito exclusivamente à engenharia.

***Item de rubrica atendido:** O aluno estruturou o acesso aos dados utilizando federação de identidades e catalogação centralizada.*

6.2 Mascaramento de Dados Sensíveis

Três funções SQL (governanca.mascara_email, mascara_cpf, mascara_telefone) usam `is_account_group_member(...)` para liberar o valor real apenas a ecom_engenharia/ecom_pii_leitores, retornando o valor mascarado (ex.: `***@dominio.com`) para qualquer outro leitor. Essas funções são aplicadas como colunas MASK na materialized view `gold.dim_clientes`.

O isolamento por região (row-level security) foi originalmente desenhado como uma cláusula ROW FILTER nativa na própria materialized view — porém, durante a execução real, o motor do Lakeflow desta versão rejeitou essa sintaxe em MATERIALIZED VIEW (PARSE_SYNTAX_ERROR ... near 'FILTER', evidência reproduzida na seção 10). A solução adotada — e prevista como plano de contingência desde o desenho da arquitetura — foi mover a aplicação do filtro para uma **view comum** dedicada, `gold.vw_clientes_regional`, que reaplica tanto as funções de máscara quanto `governanca.filtro_uf(uf)` como predicado de WHERE. O resultado funcional observado pelo consumidor final é equivalente: um analista fora do Sudeste/GO/DF não enxerga linhas de outras regiões, e nenhum leitor sem privilégio vê PII em texto claro.

***Item de rubrica atendido:** O aluno aplicou técnicas de mascaramento de dados ou isolamento de datasets sensíveis.*

7. Otimização e Performance

7.1 Z-Order, Liquid Clustering e Deletion Vectors

As tabelas Delta de maior volume de leitura seletiva usam **Z-Order** — seja via `pipelines.autoOptimize.zOrderCols` (aplicado automaticamente pelo Lakeflow em `silver.clientes` por `cliente_id` e em `silver.pedidos` por `cliente_id`, `pedido_ts`), seja explicitamente via `OPTIMIZE ... ZORDER BY (cliente_id, pedido_ts)` na tabela de auditoria. As materialized views da camada Gold usam a técnica mais recente de **liquid clustering** (`CLUSTER BY`), que elimina a necessidade de reescrever o Z-Order manualmente a cada novo padrão de consulta. Todas as tabelas de CDC (`silver.clientes`, `silver.pedidos`, `silver.produtos`) e a tabela de auditoria têm `delta.enableDeletionVectors = true`, permitindo que `DELETE/MERGE` marquem linhas como removidas sem reescrever fisicamente o arquivo Parquet inteiro — um ganho de desempenho essencial em tabelas atualizadas com alta frequência via CDC.

Item de rubrica atendido: O aluno aplicou Z-Order, vetores de exclusão ou técnicas de ordenação avançada.

7.2 Autoscale: Restrição da Free Edition e Configuração de Referência

O workspace utilizado (Databricks Free Edition) disponibiliza **exclusivamente computação serverless** — não é possível provisionar clusters clássicos (e, portanto, autoscale clássico) nesta camada gratuita. A computação serverless do Databricks já realiza dimensionamento automático e transparente de recursos internamente (o operador não escolhe o tamanho do cluster, apenas consome capacidade elástica gerenciada pela plataforma), o que tecnicamente satisfaz a necessidade de "atender à demanda" sem intervenção manual.

Para deixar explícita a compreensão de como o dimensionamento automático seria configurado em um ambiente com clusters tradicionais, o arquivo `docs/workflow/pipeline_classico_autoscale.json` documenta a configuração equivalente do mesmo pipeline com um cluster clássico: `autoscale: {min_workers: 1, max_workers: 5, mode: ENHANCED}`. Este arquivo é puramente documental/comparativo — não é referenciado pelo `databricks.yml` e não foi implantado, por não ser executável neste workspace.

Item de rubrica atendido: O aluno configurou a infraestrutura com dimensionamento automático (Autoscale) para atender à demanda — na modalidade serverless disponível, com configuração de referência documentada para o modelo clássico.

7.3 Checkpoints e Watermark

A streaming table `gold.pedidos_por_hora` agrega pedidos por janela de uma hora e canal, usando `WATERMARK pedido_ts DELAY OF INTERVAL 2 HOURS` para tolerar até duas horas de atraso na chegada dos eventos antes de fechar definitivamente uma janela — essencial para não descartar dados legítimos que chegam fora de ordem, sem deixar o estado da agregação crescer indefinidamente. Um segundo cenário de streaming, fora do pipeline declarativo (`src/operacao/demo_checkpoint_autoloader.py`), demonstra o uso "clássico" do Auto Loader via Structured Streaming direto em notebook Python, com `cloudFiles.schemaLocation`, `checkpointLocation` (persistido em um volume dedicado), `withWatermark + dropDuplicatesWithinWatermark` e `trigger(availableNow=True)` para uma carga incremental "batch-like" idempotente.

Item de rubrica atendido: O aluno garantiu a continuidade no processamento de fluxos através de Checkpoints e marcas d'água adequadas.

8. Streaming Tables, Materialized Views e Views Comuns

O projeto emprega deliberadamente os três tipos de dataset do Lakeflow, cada um onde é semanticamente correto:

Tipo	Onde é usado	Por quê
Streaming table	Todas as tabelas de bronze; eventos validados e tabelas de silver e gold dependentes de bronze	Por ser a origem; menor custo de manutenção
Materialized view	gold.vendas_diarias_categoria, funil_status_pedidos, clientes_regional, pedidos_enriquecidos (temporária); gold.vw_clientes_regional	Recomputação otimizada incremental de agregações; liquid clustering
View comum (não materializada)	pedidos_enriquecidos (temporária); gold.vw_clientes_regional	Reaproveitada em várias MVs; camada de segurança reativa

A escolha de `gold.vw_clientes_regional` como view comum (e não materializada) é deliberada: como ela aplica controle de acesso (row filter), precisa ser reavaliada a cada consulta, no contexto de quem está perguntando — o que uma materialized view, por definição, não garante da mesma forma.

Item de rubrica atendido: O aluno diferenciou e aplicou corretamente tabelas de streaming, views materializadas e views comuns.

9. Publicação de Datasets e Dependência Externa

A camada Gold depende de uma tabela mantida **fora do pipeline declarativo** — `referencia.categorias` — carregada por um MERGE manual no notebook de setup, e apenas *lida* (nunca escrita) pelo Lakeflow. Essa tabela simula um cenário realista de integração entre um Data Lakehouse e um sistema de referência externo (ex.: um ERP ou catálogo de produtos mantido por outra equipe), sem que o pipeline DLT tenha qualquer controle sobre seu ciclo de vida. A view temporária `pedidos_enriquecidos` junta `silver.pedidos`, `silver.produtos` e essa dependência externa antes de alimentar as três materialized views de negócio, demonstrando a publicação correta dos datasets finais com uma dependência explícita e rastreável (ver linhagem, seção 11).

Item de rubrica atendido: O aluno publicou corretamente os datasets finais com as devidas configurações de dependências externas.

10. Adaptações Técnicas Realizadas Durante a Implementação

O projeto foi implantado e executado de fato no workspace, e não apenas escrito em teoria. Isso expôs um conjunto de comportamentos reais do motor do Lakeflow que exigiram correções iterativas, sempre preservando a intenção original de cada requisito (nenhuma Expectation, CDC, watermark, técnica de otimização ou controle de governança foi removido para simplesmente "fazer passar" a execução). A tabela abaixo resume as adaptações mais relevantes, cada uma validada contra um erro real reproduzido em produção:

#	Sintoma observado	Causa raiz	Correção aplicada
1	PRINCIPAL_DOES_NOT_EXIST em GRANTS	GRANTS criados via CLI eram de workspace em vez de catálogo	Grupos de contas como grupos de conta pela UI (Identity and access)
2	PARSE_SYNTAX_ERROR em CHECK constraint inline não suportado em CREATE TABLE		MODIFY TABLE TABLE ... ADD CONSTRAINT
3	PARSE_SYNTAX_ERROR near 'FILTER' FILTER não suportado em MATERIALIZED VIEW		Utilizar VIEW em vez de material view comum (gold.vw_clientes_regional)
4	UNRESOLVED_ROUTINE ao resolver catálogo padrão de resolução ≠ catálogo		Funções MASK qualificadas com \${catalogo}.governanca.*
5	PARSE_SYNTAX_ERROR near 'WATERMARK' fora de ordem (depois do WHERE)		WATERMARK reposicionado logo após o FROM STREAM(...)
6	DELTA_SOURCE_TABLE_IGNORE_STREAMING lendo de tabela alvo de MERGE CDC		Excluída para a streaming table de eventos validados (append-only)
7	Schema incompatível (BIGINT vs INT/LOGICAL) START_AT herda o tipo da coluna do SEQUENCE		SEQUENCE criada como vigente_desde_evento_seq BIGINT
8	DELTA_CDC_NON_CONSTANT_ARGUMENTS changes()/RESTORE exigem literal		Verificação colocada em célula Python e interpolada como literal
9	unknown field: event_log no bundle	CLI desta versão não suporta publicar eventos via DML	Utilizar a função event_log(TABLE(...)) diretamente
10	UNRESOLVED_COLUMN expectation LATERAL explode(...) AS x só nomeia a relação		Substituir por LATERAL VIEW explode(...) AS x

Histórico completo de commits e mensagens de erro disponível no repositório GitHub (ver seção 13).

11. Evidências de Linhagem de Dados no Unity Catalog

O Unity Catalog captura automaticamente a linhagem em nível de tabela para toda consulta executada nele, exposta via `system.access.table_lineage`. A consulta abaixo, executada após a conclusão do pipeline, confirma que toda a cadeia bronze → silver → gold, incluindo a dependência externa e a tabela de auditoria, está corretamente rastreada:

```
SELECT source_table_catalog, source_table_schema, source_table_name,  
       target_table_catalog, target_table_schema, target_table_name, event_time  
FROM system.access.table_lineage  
WHERE source_table_catalog = 'ecommerce_lakehouse' OR target_table_catalog = 'ecommerce_lakehouse'  
ORDER BY event_time DESC LIMIT 200
```

Origem	Destino
(arquivos no volume de landing)	bronze.clientes_raw / pedidos_raw / produtos_raw
bronze.clientes_raw	silver.clientes_eventos_validos
bronze.pedidos_raw	silver.pedidos_eventos_validos
bronze.pedidos_raw	silver.pedidos_quarentena
bronze.produtos_raw	silver.produtos_eventos_validos
silver.clientes_eventos_validos	silver.clientes
silver.pedidos_eventos_validos	silver.pedidos
silver.pedidos_eventos_validos	gold.pedidos_por_hora
silver.produtos_eventos_validos	silver.produtos
referencia.categorias (dependência externa)	gold.clientes_valor / funil_status_pedidos / vendas_diarias_categoria
silver.clientes, silver.produtos, silver.pedidos	gold.clientes_valor / funil_status_pedidos / vendas_diarias_categoria
silver.clientes	gold.dim_clientes
silver.pedidos	auditoria.pedidos_snapshot
monitoramento.vw_event_log	monitoramento.vw_qualidade_expectations

A linhagem confirma visualmente (e é reproduzível na aba *Lineage* do Catalog Explorer para qualquer uma dessas tabelas) que nenhuma materialized view de negócio depende diretamente dos dados brutos de bronze — toda leitura de negócio passa obrigatoriamente pela camada de validação/CDC de silver, e a dependência externa (`referencia.categorias`) está corretamente registrada como uma origem que não deriva de nenhum outro dataset do pipeline.

12. Análise das Métricas de Qualidade e Impacto das Otimizações

12.1 Qualidade de dados por regra e por dataset

A view `monitoramento.vw_qualidade_expectations` agrega os eventos `flow_progress` do event log do pipeline, extraíndo o detalhamento por Expectation. Os resultados, coletados após a carga de 2 lotes (500 clientes, 40 produtos, 4.000 pedidos "CRIADO" + 800 mudanças de status = 4.800 eventos de pedido, 571 eventos de cliente):

Dataset	Regra	Aprovados	Reprovados
silver.clientes_eventos_validos	cpf_valido	563	8
silver.clientes_eventos_validos	email_valido	562	9
silver.pedidos_eventos_validos	cliente_id_valido	4767	33
silver.pedidos_eventos_validos	pedido_id_valido	4767	33
silver.pedidos_eventos_validos	preco_valido	4769	31
silver.pedidos_eventos_validos	quantidade_valida	4765	35
silver.pedidos_eventos_validos	status_valido	4778	22
gold.vendas_diarias_categoria	receita_nao_negativa	38665	0

Interpretação: a taxa de reprovação em pedidos gira em torno de 0,6%-0,7% por regra individual (ex.: $33/4800 \approx 0,69\%$ para `cliente_id_valido`), mas como cada evento inválido pode violar mais de uma regra simultaneamente, o total de eventos únicos em quarentena (154) é menor que a soma das reprovações por regra ($33+33+31+35+22 = 154$ — neste caso as violações não se sobrepuseram, o que indica que o gerador de dados produziu majoritariamente um tipo de defeito por registro inválido). Já em clientes, a taxa de CPF/e-mail inválido ($\approx 1,4\%-1,6\%$ dos eventos) é consistente com os $\sim 3\%$ de defeito configurados no gerador, considerando que nem todo evento de cliente é uma inserção nova (parte são updates, que não recebem a mesma injeção de erro). A expectation de receita não-negativa na camada Gold teve **zero reprovações em 38.665 avaliações** — evidência de que a regra de negócio "pedidos cancelados não contam receita" (filtrada antes da agregação) é suficiente para impedir valores negativos cheguem à materialized view final.

12.2 Quarentena de pedidos

Regra violada	Quantidade
quantidade_invalida	35
pedido_id_nulo	33
cliente_id_nulo	33
preco_invalido	31
status_invalido	22

Interpretação: do total de 4.000 pedidos efetivamente criados (excluindo as 800 mudanças de status de pedidos preexistentes, que não passam pela validação de criação), 154 foram para quarentena — **3,85%**, próximo da taxa de defeito de $\sim 4\%$ configurada no gerador de dados sintéticos. A distribuição das violações é razoavelmente equilibrada entre as seis regras de negócio, sem nenhuma causa dominante isolada, o que é coerente com uma injeção de defeito uniforme e aleatória (e não um problema sistêmico de uma única fonte de dados).

12.3 Duração das atualizações do pipeline

As duas atualizações finais e bem-sucedidas do pipeline (refresh do lote 1 e do lote 2 de dados) levaram **138,8 s** e **107,5 s**, respectivamente — tempos consistentes com o volume de dados processado (milhares de linhas, não milhões) e o custo fixo de inicialização de um cluster serverless para cada atualização. **Interpretação do impacto das otimizações:** como o pipeline roda em modo `continuous: false` e `serverless`, o tempo de execução é dominado pelo tempo de provisionamento do cluster serverless e pela reotimização incremental (Z-Order/liquid clustering) das `materialized views` a cada atualização, não pelo volume de dados em si — o que explica por que ambas as atualizações, mesmo processando lotes de tamanhos diferentes (3.000 vs. 1.800 eventos de pedido aproximadamente), tiveram durações da mesma ordem de grandeza.

12.4 Impacto do OPTIMIZE + VACUUM na compactação física

Na tabela de auditoria (`auditoria.pedidos_snapshot`), a sequência de operações transacionais (`UPDATE`, `DELETE`, `MERGE`, `DELETE` acidental, `RESTORE`) deixou a tabela fragmentada em **2 arquivos físicos** antes da manutenção. Após `OPTIMIZE ... ZORDER BY (cliente_id, pedido_ts)` seguido de `REORG TABLE ... APPLY (PURGE)` e `VACUUM`, a tabela foi compactada para **1 único arquivo** (`numFiles = 1`, 51.126 bytes). **Interpretação:** esse é exatamente o comportamento esperado do *small-file problem* do Delta Lake sendo corrigido — cada operação transacional (`UPDATE`, `DELETE`, `MERGE`) tende a gerar novos arquivos de dados pequenos; sem compactação periódica, o número de arquivos cresceria monotonicamente a cada ciclo de manutenção, degradando o desempenho de leitura (mais arquivos pequenos = mais overhead de I/O e de listagem de metadados por consulta). O ganho medido aqui — 2 para 1 arquivo em uma tabela pequena — é proporcionalmente pequeno, mas a mesma técnica aplicada às tabelas de produção (`silver.pedidos`, com milhares de eventos por dia) tem impacto crescente e cumulativo ao longo do tempo, especialmente combinada ao Z-Order, que também melhora o *data skipping* (menos arquivos precisam ser lidos para satisfazer um filtro por `cliente_id` ou intervalo de `pedido_ts`).

12.5 Volume de dados por tabela (estado final)

Tabela	Linhas
bronze.clientes_raw	571
bronze.pedidos_raw	4.800
bronze.produtos_raw	46
silver.clientes	544
silver.pedidos	3.873
silver.produtos	39
silver.pedidos_quarentena	154
gold.vendas_diarias_categoria	1.833
gold.funil_status_pedidos	5
gold.clientes_valor	476
gold.dim_clientes	476
gold.pedidos_por_hora	2.786
referencia.categorias	8
auditoria.pedidos_snapshot	3.873

Interpretação: a redução de 571 eventos de cliente para 544 linhas em `silver.clientes` reflete o efeito esperado do CDC SCD Type 2 — cada evento de UPDATE gera uma nova versão histórica (aumentando linhas), mas o evento de DELETE ("fora de ordem", proposital no gerador) e o evento de atualização atrasado foram corretamente descartados pela garantia de sequenciamento do `SEQUENCE BY evento_seq`, e não geraram linhas espúrias. Em pedidos, o SCD Type 1 aplicado sobre 4.646 eventos válidos resultou em 3.873 linhas finais — exatamente o esperado, já que as 800 mudanças de status atualizam (não inserem) uma linha já existente.

Item de rubrica atendido: *O aluno interpretou as métricas do pipeline e o impacto das otimizações no desempenho global.*

13. Conclusão

O projeto demonstrou, com execução real e verificável em um workspace Databricks, a implementação completa de um Data Lakehouse no padrão Medallion, cobrindo ingestão escalável via Auto Loader, orquestração declarativa com Lakeflow Declarative Pipelines, garantias transacionais ACID do Delta Lake (incluindo time travel e recuperação de estados anteriores), qualidade de dados via Expectations com isolamento explícito de registros inválidos, captura de mudanças (CDC) com estratégias SCD1/SCD2 apropriadas a cada entidade, governança centralizada com mascaramento e isolamento de PII via Unity Catalog, otimização de performance via Z-Order/liquid clustering/deletion vectors, e processamento contínuo com checkpoints e watermark. As limitações do ambiente gratuito (computação exclusivamente serverless) foram identificadas, documentadas e contornadas sem comprometer a demonstração de nenhuma competência da rubrica, e cada erro real encontrado durante a implantação foi corrigido preservando a intenção original do requisito técnico, com evidência documentada.

13.1 Repositório e Reprodutibilidade

O código-fonte completo, o histórico de commits documentando cada correção, o README.md com instruções de execução e o arquivo de evidências brutas (EVIDENCIAS.md) estão publicados em:

https://github.com/LuisFelipeMTCS/ecommerce_lakehouse